

DOCUMENTATION TECHNIQUE

Firmware ESP8266 et passerelle USB

Acquisition des capteurs DHT22, MQ-2 et PIR, alarmes locales, affichage OLED et transmission des mesures vers la plateforme par USB série.

Projet	Sentinel-X — Workshop EPSI Bac+4, consortium AetherCorp
Composants	<code>firmware/</code> (C++ / PlatformIO) et <code>scripts/esp_serial_bridge.py</code>
Public	Développeurs IoT, intégrateurs et jury technique
Version	Octobre 2026 — état du dépôt à la date de génération

Sommaire

- 01** Rôle et périmètre
Place du firmware dans la chaîne Sentinel-X

- 02** Matériel et câblage
Carte, capteurs, sorties, brochage

- 03** Architecture logicielle
Couches, ports et adaptateurs, fichiers

- 04** Logique de fonctionnement
Cadencement, préchauffage, seuils, alarmes

- 05** Interfaces locales
Écran OLED et contrat série USB

- 06** Passerelle USB vers MQTT
Validation, format JSON, robustesse

- 07** Configuration
Constantes et options de compilation

- 08** Compilation, tests et CI
Tests natifs, doubles matériels, pipeline

- 09** Limites connues
Ce que le firmware ne garantit pas

1 Rôle et périmètre

Le firmware s'exécute sur l'unique microcontrôleur **ESP8266 (NodeMCU v2)** du boîtier Sentinel-X. Il mesure l'environnement, décide localement des alarmes physiques et affiche un état lisible sur l'écran OLED. Il publie un instantané complet de son état sur le port série USB toutes les secondes.

Cette version ne contient **ni pila Wi-Fi ni client MQTT**. Les mesures rejoignent la plateforme par une passerelle Python (`scripts/esp_serial_bridge.py`), exécutée sur le PC Serveur Local. Elle lit le port série, valide chaque bloc et republie les mesures sur le broker MQTT local consommé par le backend.

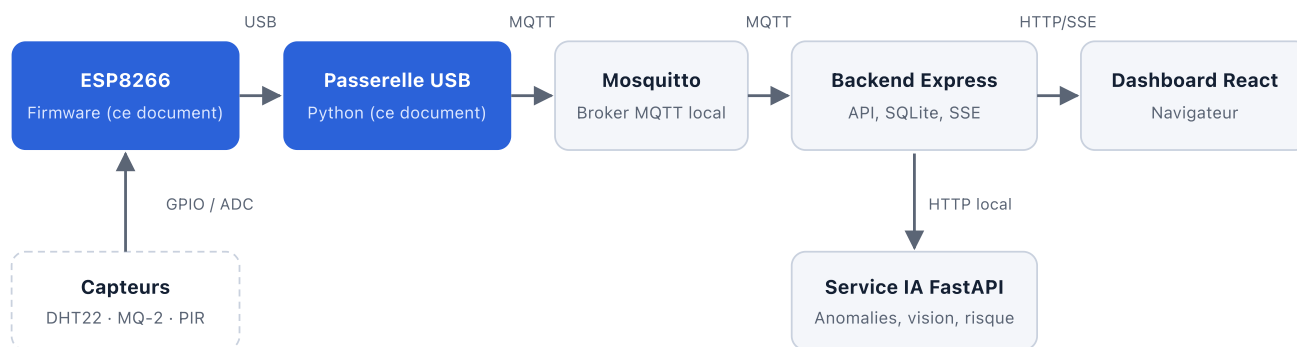


Figure 1. Chaîne de mesure Sentinel-X. Les éléments en bleu sont décrits dans ce document.

À SAVOIR

Le firmware reste **maître de ses alarmes** : LEDs et buzzer réagissent sans réseau ni PC. La passerelle est en lecture seule ; elle n'envoie jamais d'ordre à la carte.

2 Matériel et câblage

La configuration matérielle est centralisée dans `firmware/include/sentinel_config.h`. Les broches ci-dessous sont les valeurs par défaut. Toutes les sorties sont **actives à l'état haut**, pour des LEDs externes avec résistance série.

Élément	Broche NodeMCU	GPIO	Rôle et remarques
DHT22 (température, humidité)	D5	14	Type configurable (<code>DHT_TYPE</code>). Lecture toutes les 2 s minimum.
PIR HC-SR501 (présence)	D6	12	Entrée numérique ; calibration logicielle de 45 s au démarrage.
MQ-2 (gaz, fumées)	A0	ADC	Valeur brute 0–1023 (moyenne de 8 lectures). Pas une concentration en ppm.
Buzzer	D7	13	Actif pendant une alarme.
LED verte (système OK)	D0	16	Allumée quand tout est prêt, valide et sans alarme.
LED rouge (alarme)	D4	2	Partage la broche de la LED intégrée, dont la polarité peut être inversée.

Élément	Broche NodeMCU	GPIO	Rôle et remarques
LED orange (avertissement)	D8	15	Doit rester à l'état bas au démarrage (contrainte de boot de l'ESP8266).
OLED SSD1306 128x64 (SDA)	D2	4	Bus I2C à 100 kHz, adresse 0x3C, affichage tourné de 180°.
OLED SSD1306 (SCL)	D1	5	Présence vérifiée par acquittement I2C.

ATTENTION

La sortie analogique du MQ-2 alimenté en 5 V peut dépasser la plage de l'entrée A0. Vérifier le pont diviseur du module ou en ajouter un. D8/GPIO15 ne doit jamais être tiré vers le haut au démarrage, sinon la carte ne boote pas.

Bibliothèques (déclarées dans `platformio.ini`) : **Adafruit GFX**, **Adafruit SSD1306** et **DHT sensor library**. La cible est `env:nodemcu-v2` (plateforme `espressif8266`, framework Arduino, moniteur série à 115 200 bauds).

3 Architecture logicielle

Le code suit une séparation **ports et adaptateurs** : la logique métier ne connaît que des interfaces abstraites, et les pilotes matériels les implémentent. On peut ainsi compiler et tester la logique sur un ordinateur, avec des doubles matériels.

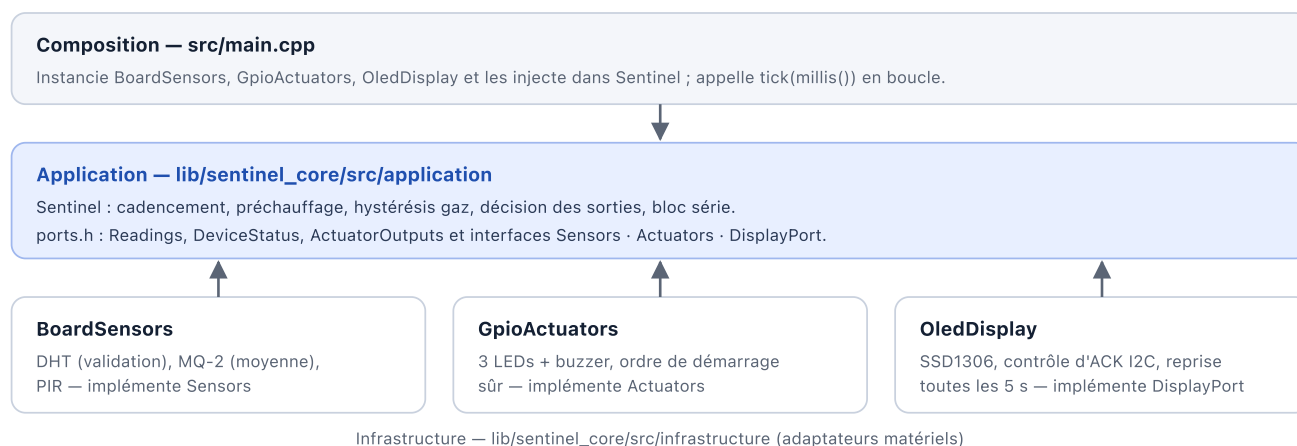


Figure 2. Couches du firmware. Les flèches indiquent le sens des dépendances : les adaptateurs dépendent des ports, jamais l'inverse.

Fichier	Responsabilité
<code>include/sentinel_config.h</code>	Configuration canonique : broches, type DHT, seuils, intervalles, préchauffages, assertions de compilation. <code>config.h</code> n'est qu'un alias.
<code>src/main.cpp</code>	Point d'entrée Arduino et composition des objets (<code>setup()</code> , <code>loop()</code>).
<code>application/ports.h</code>	Structures de données et interfaces abstraites.

Fichier	Responsabilité
application/sentinel.{h,cpp}	Orchestrateur non bloquant ; toute la logique de décision.
infrastructure/board_sensors.*	Lecture et validation DHT, moyenne MQ-2, lecture PIR.
infrastructure/gpio_actuators.*	Initialisation sûre des sorties et application des états.
infrastructure/oled_display.*	Rendu de l'écran, détection et reconnexion de l'OLED.
tests/	Tests natifs C++ (test_firmware.cpp), doubles Arduino/DHT/I2C (stubs/), lanceur run_native.py .

4 Logique de fonctionnement

4.1 Boucle non bloquante

`Sentinel::tick(nowMs)` est appelée à chaque passage de `loop()`. Chaque tâche a sa propre échéance calculée par différence (`now - last >= intervalle`), ce qui reste correct après le débordement de `millis()` (environ 49,7 jours).

Tâche	Période	Détail
PIR	chaque tick	Lecture numérique ; transitions journalisées (« MOUVEMENT DETECTE », « retour NORMAL »).
Gaz MQ-2	250 ms	Moyenne de 8 lectures ADC espacées de 300 µs.
DHT	2 000 ms	Première lecture 2 s après le démarrage, puis toutes les 2 s (assertion de compilation ≥ 2 s).
Sorties	chaque tick	Recalcul des LEDs et du buzzer à partir de l'état courant.
OLED	300 ms	Rafraîchissement de l'écran.
Bloc série	1 000 ms	Instantané complet pour la passerelle.

4.2 Préchauffage et validité des mesures

- **PIR** : ignoré pendant **45 s** (calibration du HC-SR501), présence forcée à faux.
- **MQ-2** : alerte inhibée pendant **60 s** (chauffe). La fin de chauffe est mémorisée et survit au débordement de `millis()` ; un nouvel appel à `begin()` la réinitialise.
- **DHT** : une lecture est valide si température et humidité sont finies, et dans les plages $-40\dots80$ °C et $0\dots100$ %. Sinon les deux valeurs sont effacées (**NAN**) : une ancienne mesure n'est jamais réutilisée derrière une erreur.

4.3 Seuils de gaz avec hystérésis

L'alerte gaz passe à vrai quand la valeur moyenne atteint **600**, et revient à faux quand elle redescend à **550** ou moins (bornes incluses). L'écart évite le clignotement autour du seuil. Les deux seuils sont vérifiés à la compilation ($0 \leq \text{OFF} < \text{ON} \leq 1023$).

4.4 Décision des sorties

Les sorties sont recalculées à chaque tick selon la priorité suivante : rouge, puis orange, puis verte.

```
alarmActive    = gasAlert OR motionDetected
warningActive  = NOT alarmActive AND (NOT systemReady OR NOT climateValid)
systemOK       = systemReady AND climateValid AND NOT alarmActive

LED rouge = alarmActive · buzzer = alarmActive
LED orange = warningActive · LED verte = systemOK
```

Situation	Rouge	Orange	Verte	Buzzer
Démarrage (préchauffage en cours)	—	●	—	—
Erreur DHT, sans alarme	—	●	—	—
Surveillance normale	—	—	●	—
Mouvement détecté (après 45 s)	●	—	—	●
Alerte gaz (après 60 s)	●	—	—	●

À SAVOIR

Une erreur DHT allume l'avertissement mais **ne désactive pas** les alarmes gaz et mouvement. Une température élevée, à elle seule, ne déclenche pas le buzzer : l'analyse environnementale relève du service IA.

5 Interfaces locales

5.1 Écran OLED

Au démarrage, un écran d'accueil « SENTINEL X — SECURITY SYSTEM — BOOTING... » s'affiche. Ensuite, l'écran présente :

- un bandeau avec l'état global : OK, ... (préchauffage), ERR (DHT invalide) ou AL! (alarme) ;
- la température et l'humidité en grands caractères, ou --.-C / --% si la mesure est invalide ;
- l'état du gaz (WARM, NORMAL, DANGER), sa valeur brute et une barre graphique avec le repère du seuil d'alerte ;
- l'état du PIR (PIR..., PIR OK, MOVE!) ;
- une ligne de message clignotante en alarme (!!! GAS ALERT !!!, !! MOTION ALERT !!, !! GAS + MOTION !!), sinon STARTING..., DHT SENSOR ERROR ou MONITORING.

La présence de l'écran est vérifiée par un acquittement I2C, à l'initialisation puis à chaque rafraîchissement. Sans écran, le firmware le signale sur le port série, retente une connexion toutes les 5 s et **continue de gérer les alarmes**.

5.2 Contrat série USB

Toutes les secondes, le firmware écrit un bloc délimité à **115 200 bauds**. Ce format est le contrat d'interface avec la passerelle : toute modification doit être répercutée dans `esp_serial_bridge.py` et ses tests.

```
===== SENTINEL-X =====
Temperature : 24.4 C
Humidite    : 61.1 %
Gaz MQ-2    : 40
Etat gaz    : NORMAL
Etat PIR    : MOUVEMENT
ALARME      : ACTIVE
LED ROUGE   : ON
LED ORANGE  : OFF
LED VERTE   : OFF
=====
```

Ligne	Valeurs possibles
Temperature / Humidite	nombre à une décimale suivi de C / %, ou ERREUR
Gaz MQ-2	entier 0–1023 (valeur ADC brute)
Etat gaz	CHAUFFE, NORMAL, DANGER
Etat PIR	CALIBRATION, NORMAL, MOUVEMENT
ALARME	ACTIVE, OFF
LED ROUGE / ORANGE / VERTE	ON, OFF

Les autres lignes écrites hors bloc (démarrage, transitions PIR, alertes gaz, diagnostics DHT et OLED) servent au diagnostic humain et sont ignorées par la passerelle.

6 Passerelle USB vers MQTT

`scripts/esp_serial_bridge.py` relie le port série au broker MQTT. Elle dépend de `pyserial` et `paho-mqtt` (`scripts/requirements-esp.txt`).

6.1 Analyse et validation

- Seuls les blocs **complets** entre les deux délimiteurs sont acceptés, avec exactement les neuf champs attendus et sans doublon.
- Un bloc qui dure plus de **3 s**, une ligne de plus de **256 octets**, une valeur malformée ou hors plage invalident le bloc. L'analyseur se resynchronise sur l'en-tête suivant.

- Les lignes fragmentées par les délais de lecture USB sont réassemblées, avec des limites de taille et de temps.
- Plages vérifiées : température -40...80 °C, humidité 0...100 %, gaz entier 0...1023.

6.2 Message MQTT produit

Topic par défaut `esp8266/donnees`, QoS 0, sans rétention. `ts` est l'heure Unix de réception du bloc par le PC.

```
{ "source": "live", "device": "sentinel-x-01", "ts": 1791370000.5,
  "temperature": 24.4, "humidity": 61.1, "gas": 40, "presence": true,
  "climateValid": true, "gasReady": true, "pirReady": true, "gasAlert": false,
  "alarmActive": true, "ledRed": true, "ledOrange": false, "ledGreen": false }
```

Condition firmware	Effet sur le message
DHT en <code>ERREUR</code>	<code>climateValid=false</code> ; température et humidité omises
Gaz en <code>CHAUFFE</code>	<code>gasReady=false</code> ; valeur de gaz omise
PIR en <code>CALIBRATION</code>	<code>pirReady=false</code> ; présence omise
Tous les capteurs indisponibles	seuls les huit indicateurs booléens sont envoyés ; aucune valeur fictive

6.3 Robustesse

- **Sélection du port** : sans `--port`, choix automatique uniquement s'il existe un seul port USB plausible (ports Bluetooth exclus). La sélection est refaite après une déconnexion, même si Windows change le numéro COM. Un port imposé n'est jamais remplacé.
- **Reconnexion** automatique USB et MQTT. À l'ouverture du port, les octets en attente sont écartés et les lignes DTR/RTS laissées inactives.
- **Pas d'historique hors ligne** : seul le dernier bloc est conservé, pendant 2 s au plus. Un message expiré est abandonné et l'horodatage n'est jamais rafraîchi lors d'une reconnexion.
- **Diagnostic** : avertissement si le port est ouvert sans bloc valide pendant 10 s, puis au plus toutes les 30 s.

Option	Défaut	Usage
<code>--port</code>	auto	Port série imposé (<code>COM7</code> , <code>/dev/ttyUSB0</code> ...)
<code>--baud</code>	115200	Débit série
<code>--mqtt-host</code> / <code>--mqtt-port</code>	127.0.0.1 / 1883	Broker cible
<code>--mqtt-topic</code>	esp8266/donnees	Topic de télémétrie lu par le backend
<code>--device</code>	sentinel-x-01	Identifiant inclus dans chaque message
<code>--list-ports</code>	—	Liste les ports détectés et quitte
<code>--max-messages</code>	illimité	Arrêt après N envois (diagnostic)

ATTENTION

La passerelle cible un broker local sans authentification ni TLS (`scripts/mosquitto-local.conf` écoute uniquement sur `127.0.0.1`). Il ne doit pas être exposé sur le réseau.

7 Configuration

Toutes les constantes sont dans `sentinel_config.h`. Seuls `PIN_DHT` et `DHT_TYPE` peuvent être surchargés à la compilation sans modifier le fichier.

Constante	Valeur	Signification
<code>DHT_TYPE</code>	<code>DHT22</code>	Types acceptés : 11, 12, 21, 22 (assertion de compilation)
<code>GAS_THRESHOLD_ON / OFF</code>	<code>600 / 550</code>	Hystérésis de l'alerte gaz (ADC brut)
<code>DHT_INTERVAL_MS</code>	<code>2000</code>	Période minimale du DHT
<code>GAS_INTERVAL_MS</code>	<code>250</code>	Période de lecture du MQ-2
<code>DISPLAY_INTERVAL_MS</code>	<code>300</code>	Rafraîchissement OLED
<code>SERIAL_INTERVAL_MS</code>	<code>1000</code>	Période du bloc série
<code>PIR_WARMUP_MS / GAS_WARMUP_MS</code>	<code>45 000 / 60 000</code>	Calibration PIR et chauffe MQ-2
<code>OLED_ADDRESS</code>	<code>0x3C</code>	Adresse I2C de l'écran

Pour adapter un capteur dont le type et le câblage ont été vérifiés, ajouter dans `platformio.ini` (le numéro de broche est un GPIO, D5 = GPIO14) :

```
build_flags =
    -I include
    -DDHT_TYPE=22
    -DPIN_DHT=14
```

À SAVOIR

Le type configuré ne prouve pas le type physique : le firmware n'effectue aucune détection automatique. Le démarrage affiche le type, la broche et l'intervalle utilisés pour faciliter la vérification.

8 Compilation, tests et CI

```
# Compiler pour l'ESP8266 (sans téléverser)
pio run -d firmware -e nodemcu2

# Tests natifs de la logique et des adaptateurs, avec doubles matériels
python firmware/tests/run_native.py          # g++, clang++ ou Zig détecté
python firmware/tests/run_native.py --compiler /chemin/vers/zig

# Tests de la passerelle USB (aucun matériel ni broker requis)
python -m unittest discover -s scripts/tests -v
```

Le lanceur natif compile les **vraies classes** (`Sentinel`, `BoardSensors`, `GpioActuators`, `OledDisplay`) avec des doubles d'Arduino, DHT, Wire et SSD1306, en `-Wall -Wextra -Werror`. La suite s'exécute deux fois : configuration par défaut, puis surchargée (`DHT_TYPE=21`, `PIN_DHT=0`).

Cas de test natif	Vérifie
Cadence du climat	première lecture DHT à 2 s, puis période respectée
Préchauffage	alarmes inhibées puis activées capteur par capteur
Hystérésis gaz	bornes 600 / 550 incluses
Débordement de l'horloge	préchauffage et alarme corrects après le passage de <code>millis()</code> à zéro
Redémarrage	<code>begin()</code> réinitialise alarme, climat et préchauffage
Erreur climat	avertissement sans désactiver l'alarme mouvement
Adaptateur DHT	conservation des observations, rejet des valeurs invalides
Contrat série	format exact du bloc, <code>ERREUR</code> et états de préchauffage
GPIO	polarité des sorties, démarrage silencieux (D8 bas)
OLED	ACK I2C exigé, pas de « OK » avec un climat invalide

Vérification (état du dépôt à la génération)	Résultat
Tests natifs firmware	10/10 réussis dans chaque configuration
Tests de la passerelle USB	27/27 réussis
Compilation ESP8266 (nodemcu2)	réussie — RAM 35,3 % (28 932 o), Flash 27,4 % (286 115 o)

Le workflow GitHub Actions (`.github/workflows/ci.yml`, job `firmware`) exécute les tests natifs puis compile le firmware à chaque push et pull request. La passerelle est testée dans le job Python.

9 Limites connues

- **Pas de calibration** : `climateValid` indique une lecture exploitable, pas une valeur juste. Le MQ-2 reste une valeur ADC brute, sans conversion en ppm.

- **Seuils de démonstration** : 600/550 et les délais de préchauffage doivent être évalués avec le matériel réel.
- **Transmission en lecture seule** : les boutons LED du dashboard publient sur MQTT, mais le firmware USB actuel n'écoute aucune commande. Les LEDs sont pilotées localement.
- **Pas de rejeu** : une mesure perdue pendant une coupure USB ou MQTT n'est pas retransmise (QoS 0, tampon de 2 s).
- **Sécurité du transport** : le lien série et le broker local ne sont pas chiffrés ; leur protection repose sur l'isolement de la machine serveur.
- Les tests natifs valident la logique et les adaptateurs, **pas le câblage ni la précision physique** des capteurs.